

NCollection ERP Platform — Master System Design & Execution Plan

Version: 4.0

Date: July 16, 2026

Classification: Internal — Enterprise Engineering Reference

Prepared By: Architecture & Planning Team (Gemini, Claude, ChatGPT)

Purpose: The authoritative technical reference for the NCollection ERP Platform. This document defines the complete system architecture, current project state, developer personas, and an exhaustive phase-by-phase execution plan. It assumes all previously completed milestones are stable and continues from the current project state.

Table of Contents

- [1. Project Overview & Current State](#)
 - [2. System Architecture](#)
 - [3. Two-Layer Architecture Philosophy](#)
 - [4. Existing Modules & OCA Integrations](#)
 - [5. Team Structure & Personas](#)
 - [6. Development Rules & Principles](#)
 - [7. Collaboration Workflow](#)
 - [8. Phase 1: Customer Workspace — CURRENT SPRINT](#)
 - [9. Phase 2: SaaS Automation](#)
 - [10. Phase 3: ERP Enhancement & UAE Localization](#)
 - [11. Phase 4: Executive Dashboards](#)
 - [12. Phase 5: AI Platform](#)
 - [13. Phase 6: Customer Portal](#)
 - [14. Phase 7: Mobile Application](#)
 - [15. Phase 8: Platform Services](#)
 - [16. Phase 9: Marketplace](#)
 - [17. Phase 10: Enterprise Readiness](#)
 - [18. Cross-Cutting Concerns](#)
-

1. Project Overview & Current State

1.1 What NCollection ERP Is

NCollection ERP is a **commercial SaaS ERP Platform** built on top of **Odoo 19 Community Edition**, targeting small-to-medium businesses (5–100 employees) across the **UAE and GCC region**.

Critical distinction: The project is NOT an Odoo customization. Odoo is the **ERP engine**. The real product is the **SaaS platform layer** around Odoo — tenant management, subscription licensing, provisioning, white-label branding, and UAE localization. This platform will eventually serve **thousands of companies across the GCC**.

1.2 Completed Milestones (DO NOT REDESIGN)

The following milestones are **complete and stable**. They must not be recreated, redesigned, or refactored unless explicitly requested:

Milestone	Status	What Was Delivered
Docker Infrastructure	Complete	docker-compose.yml with PostgreSQL 16 + Odoo 19 containers, volume mounts, custom_addons directory
PostgreSQL	Complete	PostgreSQL 16 running in Docker, odoo role configured, persistent volume
White Label / Branding	Complete	ncollection_branding module — custom logo, login page, favicon, SCSS theme colors, browser title override
SaaS Foundation	Complete	ncollection_subscription module — Tenant, Subscription, Plan, Provisioning Job models with full ORM, chatter integration, computed fields
Organizations	Complete	ncollection.tenant model with UUID, database status tracking, onboarding stages, contact info, plan linking
Subscription Plans	Complete	ncollection.subscription.plan model with tiered pricing (monthly/yearly), user limits, currency support
Provisioning Queue	Complete	ncollection.provisioning.job model with status tracking (queued/running/done/failed), logging
Financial Reporting	Complete	OCA account_financial_report installed — General Ledger, Trial Balance, Journal Ledger, VAT Report, Open Items, Aged Partner Balance
MIS Builder	Complete	OCA mis_builder installed with custom ncollection_mis_templates (Balance Sheet, P&L)
Fresh Origin Demo	Complete	Demo company with sample data across CRM, Sales, Purchase, Inventory, HR, Projects
Dashboard Redesign	Complete	ncollection.subscription.dashboard transient model with KPI computations (total tenants, MRR, expiring subscriptions)
OCA Integration	Complete	account_financial_report + mis_builder integrated and verified
CI/CD Foundation	Complete	GitHub Actions pipeline with flake8 linting on PRs to develop and main

1.3 Current Project State — What Exists in Code

Repository: NCollection-Sys/ncollection-erp (Private GitHub)

```

ncollection-erp/
├── custom_addons/
│   ├── ncollection_branding/          Implemented (partial — pending items remain)
│   │   ├── __manifest__.py
│   │   ├── views/webclient_templates.xml (title + favicon)
│   │   ├── static/src/scss/theme_colors.scss
│   │   ├── static/src/img/
│   │   └── data/res_company_data.xml
│   └── ncollection_subscription/      Implemented (models + views + demo data)
│       ├── models/ (tenant.py, subscription.py, subscription_plan.py,
│       │           provisioning_job.py, dashboard.py)
│       ├── views/ (6 XML view files + menus)
│       ├── security/ir.model.access.csv
│       ├── data/demo_data.xml
│       └── static/src/scss/dashboard.scss
├── ncollection_core/                  Skeleton only (manifest + empty init)
├── ncollection_saas/                  Skeleton only (manifest + empty init)
├── docs/                             PRD, Roadmap, Master Context, System Design
├── .github/workflows/ci.yml           flake8 on PRs
└── docker-compose.yml                 PostgreSQL 16 + Odoo 19

```

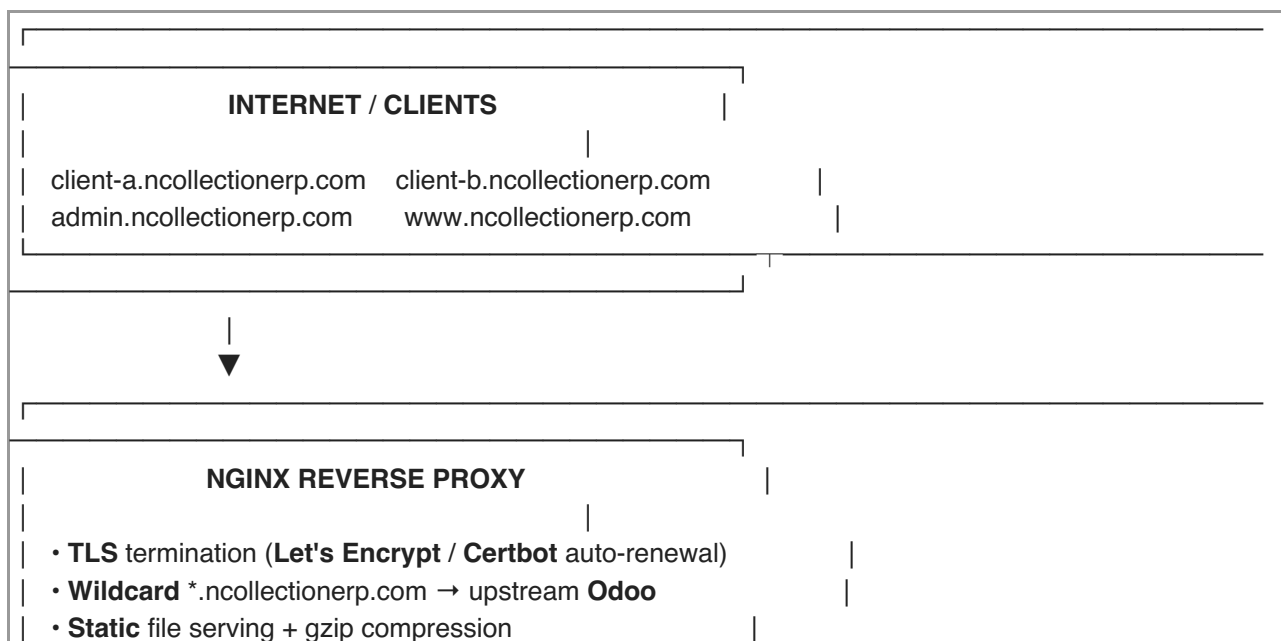
1.4 Current Sprint: Customer Workspace

The team must focus **exclusively** on the **Customer Workspace** phase. Everything else is lower priority and must not be started until Customer Workspace is complete, tested, and stable.

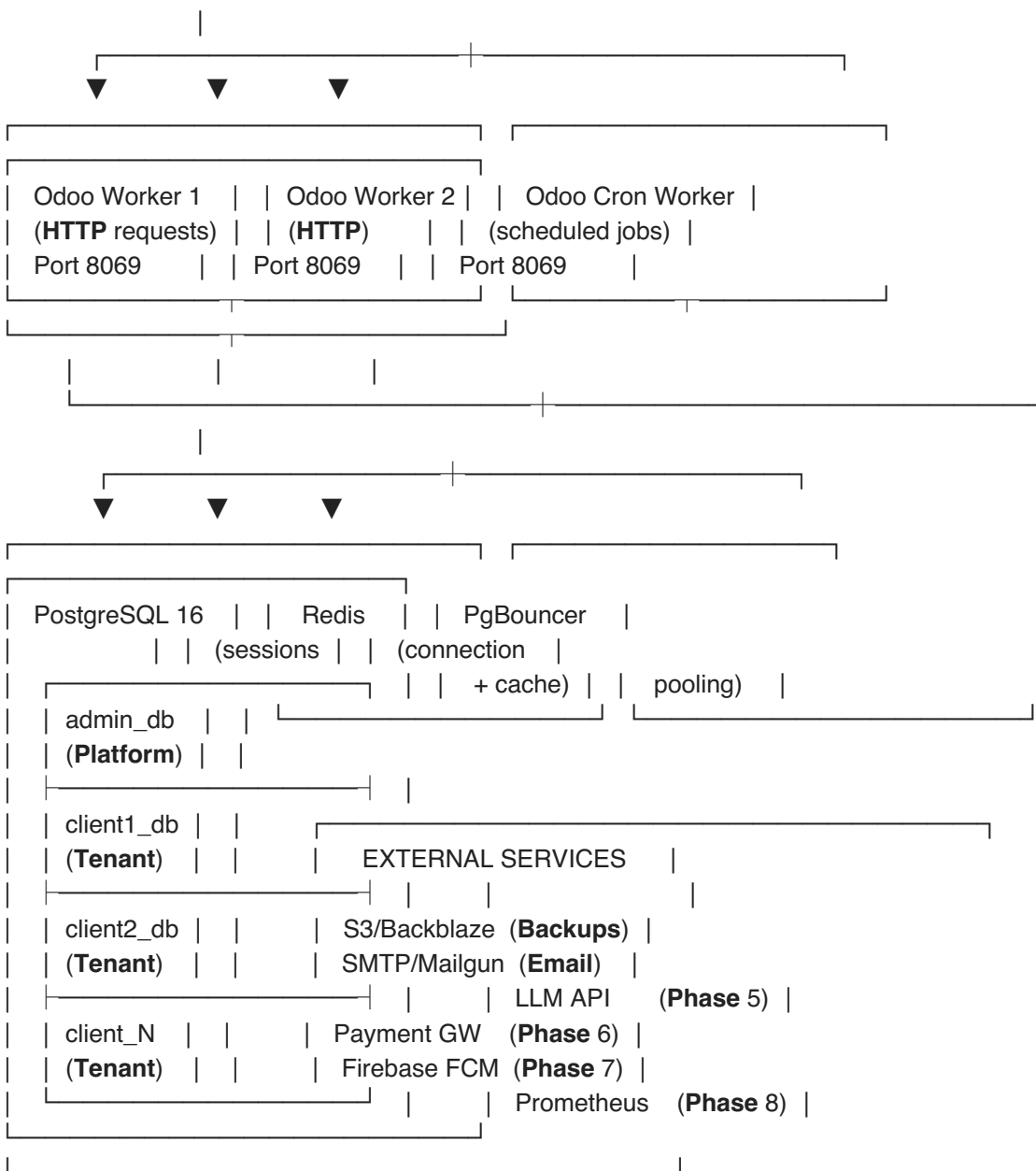
2. System Architecture

2.1 Architecture Overview

The NCollection ERP Platform follows a **database-per-tenant** multi-tenant SaaS architecture, where a single Odoo 19 process serves multiple isolated PostgreSQL databases, each belonging to a different subscribing company.



- **Rate** limiting per **IP** / per tenant
- **WebSocket** proxy for longpolling (**port 8072**)
- X-Forwarded-For / X-Forwarded-Proto / Host headers
- Security headers (**HSTS**, **X-Frame-Options**, **CSP**)



2.2 Database-per-Tenant Isolation Strategy

Chosen Model: Database-per-Tenant (same model as Odoo.com SaaS)

Reason: This is the only model that provides complete data isolation, independent backups, independent migrations, and zero risk of cross-tenant data leakage. For a GCC-targeted commercial SaaS platform, data sovereignty and isolation are non-negotiable requirements.

Benefits:

- Complete data isolation between tenants — impossible to accidentally query another tenant's data

- Independent backup and restore per tenant without affecting others
- Independent Odoo module upgrades per tenant (can roll out changes gradually)
- Compliance-friendly — each tenant's data can be stored/deleted independently
- Follows the proven Odoo.com architecture

Risks:

- PostgreSQL connection count grows linearly with tenant count (mitigated by PgBouncer)
- Cron jobs run per-database, increasing CPU usage as tenants grow
- Schema migrations must be applied to all tenant databases

Alternatives Considered:

Alternative	Why Rejected
Multi-company (single DB)	No true isolation; risk of data leakage; cannot do independent backups; one bad migration affects everyone
Schema-per-tenant	Odoo doesn't support this natively; would require deep core modifications violating Rule 1
Row-level security	Same as multi-company; insufficient for enterprise SaaS compliance requirements

Recommendation: Continue with database-per-tenant. Invest in PgBouncer for connection pooling once tenant count exceeds 20.

2.3 Subdomain Routing Mechanism

How It Works:

1. DNS: *.ncollectionerp.com → VPS IP (wildcard A **record**)
2. Nginx: Receives request **for** clienta.ncollectionerp.com
3. Nginx: Proxies **to** Odoo **with** Host **header** preserved
4. Odoo: --db-filter=^%h\$ extracts "clienta" from hostname
5. Odoo: Routes request **to database** "clienta" (**or** "clienta_db")
6. Odoo: **Session** cookie **is** scoped **to** that **database only**

Odoo Configuration (odoo.conf):

```
[options]
; --- Multi-Tenant Routing ---
db_host = db
db_port = 5432
db_user = odoo
db_password = ${DB_PASSWORD}
db_name = False ; Allow connections to any database
db_filter = ^%h$ ; Route by subdomain (CRITICAL)
list_db = False ; SECURITY: Hide database selector completely

; --- Performance (4 vCPU / 8 GB RAM) ---
workers = 4 ; (2 × CPU_cores) + 1
max_cron_threads = 1 ; Dedicated cron worker
limit_memory_hard = 2684354560 ; 2.5 GB per worker
limit_memory_soft = 2147483648 ; 2.0 GB per worker
limit_time_cpu = 600 ; 10 min CPU time limit
limit_time_real = 1200 ; 20 min wall clock limit
limit_time_real_cron = 3600 ; 1 hour for cron jobs

; --- Security ---
admin_passwd = ${ADMIN_MASTER_PASSWORD} ; Strong, unique; disabled in prod
proxy_mode = True ; Trust Nginx X-Forwarded headers

; --- Paths ---
addons_path = /mnt/extra-addons,/usr/lib/python3/dist-packages/odoo/addons

; --- Logging ---
log_level = info
logfile = /var/log/odoo/odoo-server.log
log_handler = :INFO,werkzeug:WARNING
```

Reason for each critical setting:

- `db_filter = ^%h$` — The single most important setting for multi-tenant SaaS. It ensures each subdomain only sees its own database.
- `list_db = False` — Prevents users from seeing the database dropdown on the login page. Without this, any visitor could see all tenant database names — a serious security and privacy violation.
- `proxy_mode = True` — Required when Odoo sits behind Nginx. Without this, Odoo generates incorrect URLs (HTTP instead of HTTPS) and logs Nginx's IP instead of the client's IP.

2.4 Isolation Guarantees

Layer	Mechanism	Verification
Database	Each tenant has its own PostgreSQL database — no shared tables	<code>SELECT datname FROM pg_database</code> shows separate DBs
Application	<code>--db-filter</code> ensures each HTTP request only accesses the matched database	Test: access <code>clienta.ncollectionerp.com</code> , verify no data from <code>clientb</code> is accessible
Session	Odoo sessions are database-scoped — cookies are tied to the specific DB	Test: log into <code>clienta</code> , then visit <code>clientb</code> — must see login page, not <code>clienta</code> 's session
Filestore	Attachments stored in <code>~/.local/share/Odoo/filestore/<db_name>/</code> —	Verify directory listing shows separate folders per tenant

	physically separated	
Network	Nginx enforces subdomain routing; no URL path exposes another tenant	Test: no API call from clienta can retrieve clientb records

2.5 Docker Infrastructure (EXISTING — Completed)

Current State (already working):

```

services:
  db:
    image: postgres:16
    container_name: ncollection-db
    environment:
      POSTGRES_DB: postgres
      POSTGRES_USER: odoo
      POSTGRES_PASSWORD: odoo
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data

  odoo:
    image: odoo:19
    container_name: ncollection-odoo
    depends_on:
      - db
    ports:
      - "8069:8069"
    environment:
      HOST: db
      USER: odoo
      PASSWORD: odoo
    restart: always
    volumes:
      - odoo_data:/var/lib/odoo
      - ./custom_addons:/mnt/extra-addons

volumes:
  postgres_data:
  odoo_data:

```

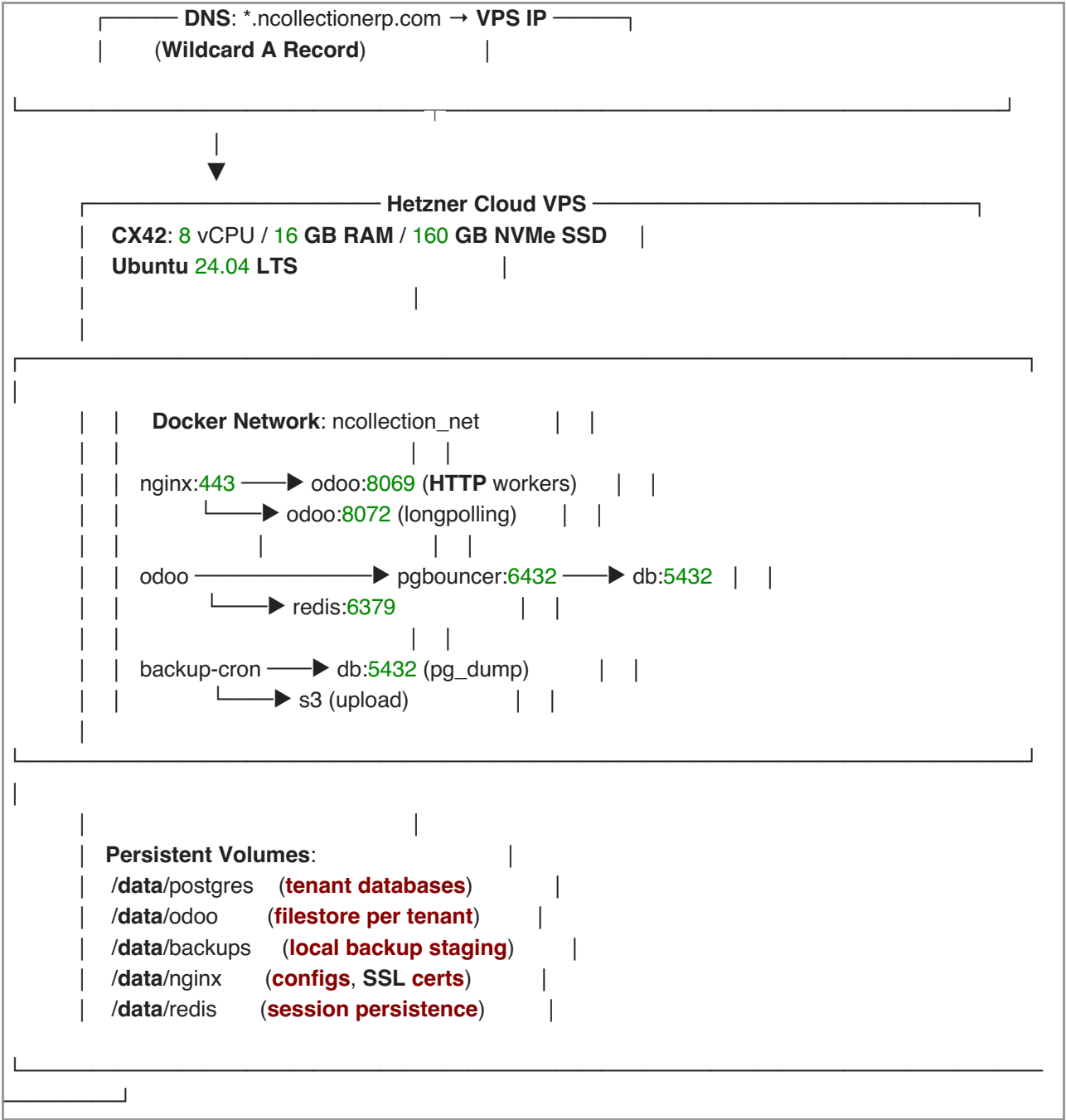
Planned Production Enhancements (Phase 2+):

Service	Purpose	When
nginx	Reverse proxy, TLS, subdomain routing	Phase 1 (P1-T05)
redis	Session store, cache	Phase 2
pgbouncer	Connection pooling (when tenants > 20)	Phase 2
backup-cron	Automated pg_dump to cloud storage	Phase 2
prometheus + grafana	Monitoring	Phase 8
certbot	SSL auto-renewal	Phase 1 (P1-T05)

2.6 Security Architecture

Concern	Mechanism	Status
TLS/HTTPS	Nginx with Let's Encrypt; HTTP→HTTPS redirect enforced	Phase 1
Database selector hidden	list_db = False in odoo.conf	Phase 1
Admin master password	Strong unique password; database management API disabled in production	Phase 1
Session security	SameSite=Lax, Secure flag, database-scoped sessions	Phase 1
Rate limiting	Nginx limit_req_zone on login endpoints	Phase 1
Firewall	UFW: only ports 80, 443, 22 open; PostgreSQL NOT exposed	Phase 2
SSH	Key-based auth only; password auth disabled	Phase 2
Secrets	.env file (not committed); Docker secrets in production	Phase 1
Audit trail	Field-level change tracking on critical models	Phase 8

2.7 Network Topology (Production Target)



3. Two-Layer Architecture Philosophy

Rule 3: Platform Layer and ERP Layer are completely different responsibilities. Never mix them.

This is the single most important architectural principle of the NCollection ERP Platform. Every developer, every code review, and every design decision must respect this boundary.

3.1 Platform Layer (NCollection SaaS)

Responsibility: The business of selling and operating ERP as a service.

Component	Description	Module
Organizations	Tenant companies, UUIDs, onboarding stages	ncollection_subscription
Plans	Subscription tiers, pricing, module licensing	ncollection_subscription
Subscriptions	Tenant↔Plan linking, billing cycle, status lifecycle	ncollection_subscription
Provisioning	Database creation, module installation, admin user setup	ncollection_subscription → ncollection_saas
Billing	Invoice generation for SaaS subscriptions	ncollection_saas (Phase 2)
Domains	Subdomain assignment, SSL management	ncollection_saas (Phase 2)
Licensing	Module visibility control per subscription	ncollection_core (Phase 1)
Monitoring	Platform health, tenant DB sizes, request metrics	ncollection_saas (Phase 8)

Accessed by: NCollection platform administrators only (never by tenant end-users).

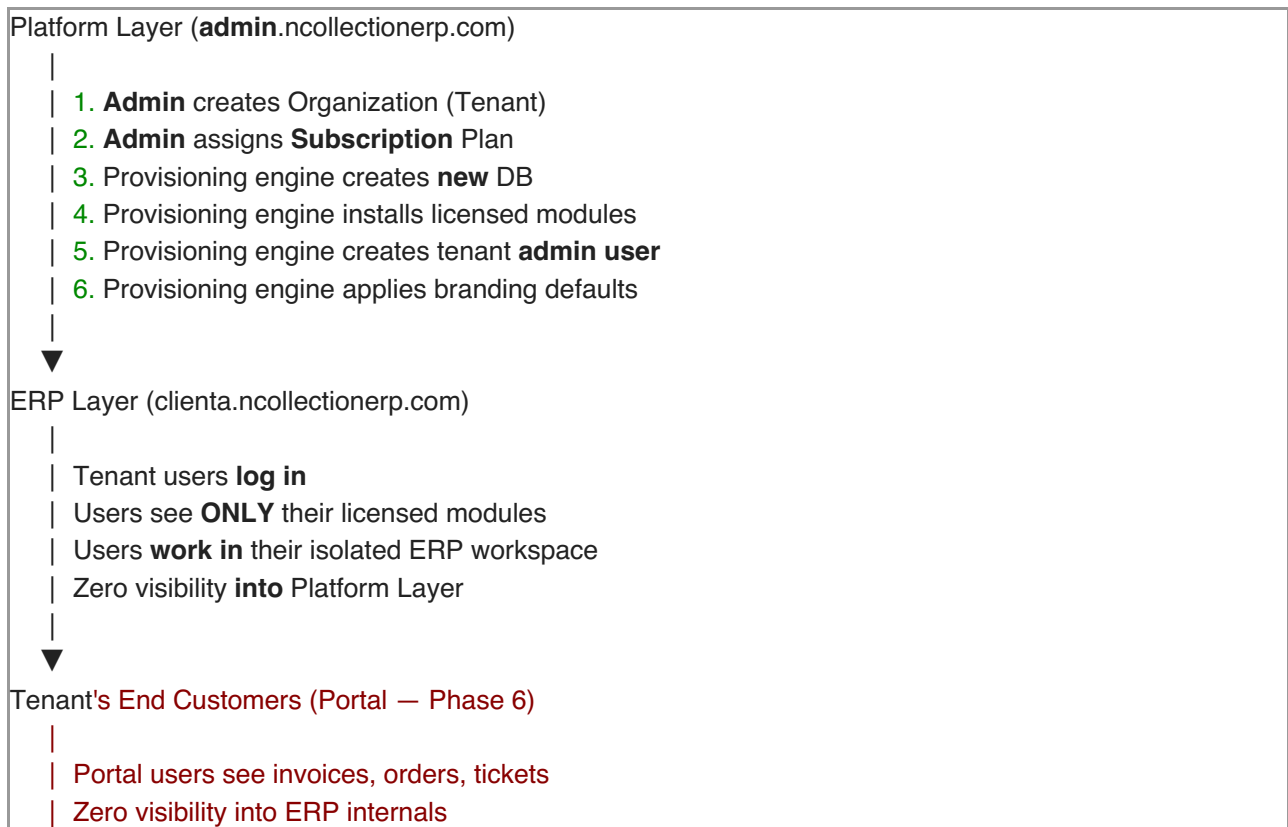
3.2 ERP Layer (Odoo 19 Community)

Responsibility: The actual business operations for each subscribing company.

Component	Odoo Module	Customization Approach
CRM	crm	XML view inheritance, Python _inherit
Sales	sale	Workflow enhancements via custom addon
Purchase	purchase	Approval workflows via custom addon
Inventory	stock	Barcode endpoints via custom addon
Accounting	account + OCA modules	UAE localization via ncollection_uae
HR	hr	Attendance, leave management via OCA
Projects	project	Standard usage

Accessed by: Tenant end-users (employees of subscribing companies).

3.3 How the Layers Interact



4. Existing Modules & OCA Integrations

4.1 Custom NCollection Modules (Existing)

Module	Status	Description	Key Models
ncollection_branding	Partial	White-label: logo, favicon, title, SCSS colors	— (template overrides)
ncollection_subscription	Core	SaaS foundation: tenants, plans, subscriptions, provisioning, dashboard	ncollection.tenant, ncollection.subscription, ncollection.subscription.plan, ncollection.provisioning.job, ncollection.subscription.dashboard
ncollection_core	Skeleton	Will hold: roles, access rights, module visibility engine	—
ncollection_saas	Skeleton	Will hold: provisioning automation, billing, domain management	—

4.2 Planned Custom Modules

Module	Phase	Description
ncollection_uae	3	UAE localization: VAT, CoA, AED, Arabic, invoice templates
ncollection_ai	5	AI platform: LLM gateway, context engine, chat widget
ncollection_portal	6	Portal redesign and customer-facing ticketing
ncollection_api	8	Public REST API with OAuth2

4.3 OCA Modules (Installed)

Module	OCA Repository	Branch	Status
account_financial_report	OCA/account-financial-reporting	19.0	Installed
mis_builder	OCA/mis-builder	19.0	Installed

4.4 OCA Modules to Evaluate Before Custom Development

Rule 2: Always search OCA before suggesting new development. Never reinvent mature OCA modules.

Before building ANY new feature, the team MUST check these OCA repositories for existing solutions:

Need	OCA Repository to Check	If Available
Audit Trail	OCA/server-tools → auditlog	Use instead of custom P8-T04
REST API	OCA/rest-framework → base_rest	Evaluate vs. custom P8-T01
Webhooks	OCA/server-tools → base_webhook	Use instead of custom P8-T02
Multi-currency	OCA/currency	Check for exchange rate providers
UAE Payroll	OCA/payroll	Use if UAE-compatible
Helpdesk/Ticketing	OCA/helpdesk	Evaluate vs. custom P6-T03
Portal enhancements	OCA/website	Check for portal improvements
Queue Job	OCA/queue → queue_job	Use for async provisioning
PDF improvements	OCA/reporting-engine	Check for QWeb report tools

Process: Before starting any task, the assigned developer must:

1. Search the relevant OCA repository for an Odoo 19-compatible module
2. If found: evaluate fit, install in dev environment, test compatibility
3. If suitable: use the OCA module and document the integration
4. If not suitable: document WHY it was rejected, then build custom
5. Log the decision in the GitHub Issue for the task

5. Team Structure & Personas

5.1 Human Development Team

[DEV-1] Backend & Infrastructure Lead

Core Skills: Python 3.12+, PostgreSQL 16, Docker/Docker Compose, Linux (Ubuntu), Nginx, CI/CD, REST API design, shell scripting, security hardening.

Responsibilities:

- Owns the entire infrastructure layer: Docker, Nginx, PostgreSQL, CI/CD, server provisioning
- Designs and implements SaaS automation: tenant provisioning, database creation, backup management, domain/SSL management
- Builds the DB routing engine (--db-filter, Nginx subdomain routing)

- Implements API layers (REST, OAuth2, mobile API optimization)
- Handles security hardening, performance tuning, and monitoring setup
- Manages deployment pipeline and server operations

Module Ownership: Infrastructure configs, ncollection_saas (provisioning), ncollection_api, monitoring.

[DEV-2] Odoo & Business Logic Specialist

Core Skills: Odoo ORM, XML views (form/tree/kanban/search), QWeb reports, Access Rights (ir.model.access, ir.rule), Odoo Accounting, Workflows, OCA module integration.

Responsibilities:

- Owns all business logic: model definitions, computed fields, constraints, state machines
- Implements module visibility engine (subscription-based menu filtering)
- Defines res.groups, access rights, and record rules for tenant role isolation
- Handles ERP enhancements: CRM, Sales, Purchase workflows
- Owns UAE localization: VAT, Chart of Accounts, currency, Arabic translations
- Implements billing engine, KPI logic, anomaly detection

Module Ownership: ncollection_subscription (business logic), ncollection_core (roles/access), ncollection_uae.

[DEV-3] Frontend & Integration Specialist

Core Skills: OWL framework, JavaScript (ES6+), QWeb templates, SCSS/CSS, responsive design, mobile development (React Native / Flutter), UI/UX design.

Responsibilities:

- Owns the branding system: ncollection_branding completion (logos, login page, About dialog, URL rewriting, email templates)
- Builds dynamic per-tenant branding (CSS variable injection)
- Develops all dashboard UIs: Customer Dashboard, CEO Dashboard, Department Dashboards (OWL + charting)
- Designs and implements the SaaS checkout flow
- Builds the customer portal UI redesign
- Creates the AI Assistant chat widget (OWL)
- Develops the mobile application

Module Ownership: ncollection_branding, all UI components, dashboard widgets, portal templates, mobile app.

5.2 AI Engineering Team

AI Agent	Role	Responsibility
ChatGPT	Chief Solution Architect	High-level architecture decisions, business logic design, OCA module evaluation, pattern recommendations
Claude	Implementation Engineer	Code generation, pair programming with developers, debugging, code review assistance
Gemini	Architecture Reviewer & Planning Assistant	Architecture review, planning documents, dependency analysis, risk assessment, timeline estimation

AI Collaboration Rules:

1. AI agents understand the current milestone before generating code
 2. AI agents verify architecture alignment with the two-layer philosophy
 3. AI agents avoid regressions and maintain Odoo 19 Community compatibility
 4. AI agents build features incrementally — never generate entire modules in one shot
 5. AI agents NEVER jump to future phases unless the current phase is complete
-

6. Development Rules & Principles

These rules are **mandatory and non-negotiable**. Every developer, every code review, and every AI-generated code must comply.

Rule 1: Never Modify Odoo Core

Reason: Modifying Odoo core files creates unsustainable technical debt. Any change to core files will be overwritten during Odoo version upgrades, causing regressions, data loss, and weeks of debugging.

Approved Extension Methods:

Method	When to Use	Example
Custom Addons	Always the primary approach	<code>ncollection_branding</code> , <code>ncollection_subscription</code>
Python <code>_inherit</code>	Extending existing models	<code>class ResCompany(models.Model): _inherit = 'res.company'</code>
XML View Inheritance	Modifying existing views	<code><template inherit_id="web.login"></code>
OWL Component Patching	Modifying frontend components	<code>patch(WebClient.prototype, { ... })</code>
SCSS Overrides	Changing visual styles	Custom SCSS in asset bundles
Controller Override	Modifying HTTP routes	<code>class CustomHome(Home): @route('/web/login')</code>

Alternatives Rejected: Direct edits to Odoo source files, monkey-patching at module load time, replacing core Python files.

Rule 2: OCA First

Reason: The Odoo Community Association (OCA) maintains 1000+ battle-tested modules. Reinventing what OCA has already built wastes development time and produces less reliable code.

Process: Before ANY new development → search OCA → evaluate → decide → document.

Rule 3: Two-Layer Separation

See [Section 3](#). Platform Layer and ERP Layer must never mix code, models, or views.

Rule 4: Upgrade Compatibility

Reason: Odoo releases a new major version annually. NCollection must be able to upgrade from Odoo 19 → 20 → 21 without rewriting custom modules.

Requirements:

- No fragile overrides that depend on internal Odoo implementation details
- Use public API methods only (`env['model'].search()`, not `self.env.cr.execute('SELECT...')` unless for

performance-critical analytics)

- Document every `_inherit` override with the reason and the Odoo method being overridden
- Pin OCA module versions in requirements files

Rule 5: Enterprise SaaS Mindset

Reason: This is not a single-company Odoo deployment. It's a multi-tenant SaaS platform that will serve thousands of companies. Every decision must consider: scalability, tenant isolation, zero downtime, data sovereignty, and operational overhead.

Checklist for every feature:

- [] Does this work correctly across 100+ tenant databases?
- [] Does this maintain tenant data isolation?
- [] Does this impact Odoo startup time or memory usage?
- [] Can this be configured per-tenant without code changes?
- [] Does this preserve upgrade compatibility?

7. Collaboration Workflow

7.1 Git Branching Strategy

```
main (production-ready, always deployable)
├── develop (integration branch — all PRs merge here)
│   ├── feature/P1-T01-docker-hardening    (DEV-1)
│   ├── feature/P1-T06-module-visibility    (DEV-2)
│   ├── feature/P1-T09-branding-completion  (DEV-3)
│   ├── hotfix/login-csrf-fix               (any DEV)
│   └── ...
```

Rules:

1. main is always deployable. Only develop merges into main via release PRs.
2. All feature branches merge into develop via reviewed PRs.
3. Feature branches follow naming: `feature/P{phase}-T{task_id}-{short-description}`.
4. Hotfix branches: `hotfix/{description}` — merge into both main and develop.

7.2 Commit Message Convention

```
[P1-T01] feat: Harden docker-compose for multi-tenant production
```

- **Add** Nginx service **with** wildcard SSL **configuration**
- **Add** Redis **for** session store
- Configure `odoo.conf` **with** `db-filter` and `list_db=False`
- **Create** `.env` file **for** secrets management

```
Closes #42
```

Format: `[P{phase}-T{id}] {type}: {description}`

Types: feat, fix, refactor, docs, test, chore, perf.

Phase 1: Customer Workspace — CURRENT SPRINT

Priority: P0 — IMMEDIATE — **THE ONLY FOCUS**

Objective: Build the foundational SaaS experience where subscribers land directly in their isolated ERP workspace, see only their licensed modules, and experience NCollection branding with zero Odoo references.

IMPORTANT: This phase builds ON TOP of the already-completed SaaS foundation. The tenant, subscription, and plan models ALREADY EXIST. This phase focuses on the CUSTOMER-FACING experience — what the tenant's employees see when they log into their workspace.

Acceptance Criteria (Definition of Done):

- [] Users can log in via company.ncollectionerp.com
- [] Users land directly in their ERP workspace (NOT a SaaS admin panel)
- [] Users see ONLY their licensed modules based on subscription plan
- [] Users CANNOT access any SaaS administration screens (Organizations, Plans, Subscriptions, Provisioning)
- [] Users CANNOT see or access any other tenant's data
- [] Basic role-based access control works within the tenant (Owner, CEO, Manager, Sales, etc.)
- [] Tenant can customize their own logo and company information
- [] All visible branding says "NCollection ERP" — zero Odoo references anywhere
- [] Customer Dashboard shows business KPIs relevant to the tenant

Phase 1 Tasks

ID	Task Name	Description	Assigned Dependencies
P1-T01	Docker Environment Hardening	Extend the EXISTING docker-compose.yml to production-grade: (1) Add Nginx service with wildcard *.ncollectionerp.com server block, TLS termination via Certbot, WebSocket proxy for longpolling on port 8072, rate limiting on /web/login, and security headers (HSTS, X-Frame-Options). (2) Create config/odoo.conf with all multi-tenant settings: db_filter=^%h\$, list_db=False, proxy_mode=True, workers=4, memory limits. (3) Create .env file for secrets (DB password, admin master password) — add .env to .gitignore. (4) Create docker-compose.dev.yml override: pgadmin service, single worker, debug mode, mounted source. (5) Add Docker health checks for all services. (6) Create docker-compose.prod.yml for production overrides. Reason: The existing docker-compose works for development but lacks the security and routing infrastructure required for multi-tenant SaaS. Benefits: Dev/staging/prod parity, secure secrets management, TLS from day one. Risks: Nginx wildcard config complexity; Certbot requires real domain for staging tests. Alternative: Use Traefik instead of Nginx (rejected: Nginx is more widely	[DEV-1] None

understood and documented for Odoo). **Recommendation:** Use Nginx with manual config generation per-tenant initially; automate in Phase 2.

Extend the EXISTING GitHub Actions CI pipeline (.github/workflows/ci.yml). Add: (1) pylint-odoo alongside flake8 for Odoo-specific linting. (2) Docker build smoke test: build the production image, start services, verify Odoo responds with HTTP 200 on http://localhost:8069/web/login. (3) Python unit test runner: start PostgreSQL in CI, run odoo-bin --test-enable -d test_db --stop-after-init -i ncollection_subscription,ncollection_core,ncollection_branding.

P1- CI Pipeline
T02 Enhancement (4) Add branch protection rules documentation (require CI pass + 1 approval for develop). **Reason:** Current CI only runs flake8, which catches syntax errors but not Odoo-specific issues or runtime failures. **Benefits:** Catch model errors, missing dependencies, and import failures before merge. **Risks:** CI test time may increase to 3–5 minutes per run. **Alternative:** Use pre-commit hooks locally (complement, don't replace CI). **Recommendation:** Implement all four enhancements; accept the longer CI time as a worthwhile tradeoff. [DEV-1] P1-T01

Complete the EXISTING skeleton modules ncollection_core and ncollection_saas. For each: (1) Create proper __init__.py with imports, (2) Update __manifest__.py with correct dependencies, (3) Create placeholder directories: models/, views/, security/, data/, static/, controllers/, wizards/, (4) Ensure all addons are importable by Odoo without errors (odoo-bin -i ncollection_core --stop-after-init exits cleanly).

P1- Addon
T03 Skeleton
Finalization **Reason:** DEV-2 and DEV-3 cannot start addon work without importable module structures in place. **Benefits:** Unblocks parallel development; establishes consistent module structure. **Risks:** Minimal — this is scaffolding. **Recommendation:** DEV-1 should complete this within the first 2 days of the sprint. [DEV-1] P1-T01

The ncollection.tenant and ncollection.subscription models ALREADY EXIST with core fields. Enhance them for the Customer Workspace: (1) Add module_ids Many2many field to ncollection.subscription.plan → ir.module.module (defines which Odoo modules this plan grants access to). (2) Add user_count computed field to tenant (counts users in the tenant's DB — may need cross-DB query or local tracking). (3) Add days_remaining computed field to subscription. (4) Add state transition methods with validation: action_activate(), action_suspend(), action_expire(), action_cancel(), action_renew(). Each transition must verify the current state is valid for the requested transition. (5) Add _track_subtype() for mail notifications on status changes. (6) Add _check_

P1- Tenant Model
T04 Enhancements constraints: end_date > start_date, max_users > 0. **Reason:** The existing models have fields but lack business logic, state transitions, and the critical module_ids field needed for module visibility. **Benefits:** Enables the Module Visibility Engine; prevents invalid state transitions; improves admin UX with [DEV-2] P1-T03

chatter notifications. **Risks:** module_ids linking to ir.module.module requires that the admin DB has the same modules installed as tenant DBs. **Alternative:** Use a text field listing module technical names instead of M2M to ir.module.module (simpler, avoids cross-DB concerns).

Recommendation: Use a text/serialized field approach for module lists (comma-separated technical names) to avoid cross-database complications.

Implement subdomain-to-database routing. (1) Configure odoo.conf with --db-filter=^%h\$. (2) Create Nginx wildcard server block: server_name *.ncollectionerp.com; proxying to Odoo with proxy_set_header Host \$host;. (3) Create development workaround: add entries to /etc/hosts (e.g., 127.0.0.1 clienta.localhost clientb.localhost admin.localhost) and Nginx config for .localhost domains. (4) Create a second test database manually to verify routing works. (5) Document the complete routing flow with diagrams. (6) Test: access clienta.localhost:8069 → see only clienta database; access clientb.localhost:8069 → see only clientb database; access admin.localhost:8069 → see only admin database. **Reason:** This is the backbone of the entire multi-tenant SaaS platform. Without working subdomain routing, nothing else in the platform functions. **Benefits:** True tenant isolation; familiar URL pattern for end-users; follows Odoo.com's proven model. **Risks:** Local development with subdomains requires /etc/hosts hacks or a local DNS resolver like dnsmasq. CI environments may struggle with subdomain testing. **Alternative:** Use URL-path-based routing instead of subdomains (rejected: less professional, more error-prone, doesn't match industry standard). **Recommendation:** Implement subdomain routing with the /etc/hosts workaround for development; automate domain management in Phase 2.

Create the core engine that controls which Odoo modules (menu items) each tenant sees. (1) Override ir.ui.menu's _visible_menu_ids() method in ncollection_core. (2) At menu load time, read the tenant's active subscription plan (from a local config record synced during provisioning — NOT a cross-database query). (3) Compare the plan's allowed modules against installed modules. (4) Remove menu items for unlicensed modules from the returned set. (5) Create ncollection.workspace.config model (installed in each tenant DB during provisioning) with fields: allowed_module_names (Text — comma-separated technical names like crm,sale,stock), plan_code, subscription_status. This record is written during provisioning and updated when the subscription changes. (6) Test: Starter plan tenant sees only CRM + Sales + Invoicing menus; Enterprise plan tenant sees all menus.

Reason: Module visibility control is the core product differentiator. Without it, every tenant sees every module regardless of what they paid for. **Benefits:** Subscription monetization; clean UX; prevents user confusion from seeing

P1- DB Routing
T05 Engine

[DEV-1] P1-T01

P1- Module
T06 Visibility
Engine

[DEV-2] P1-T04

modules they didn't purchase. **Risks:** Overriding `_visible_menu_ids` may break if Odoo 19 changes the menu loading internals. Mitigation: pin to the specific method signature and add a version check. **Alternative:** Use `res.groups` to control module access (partially works but doesn't hide menus — users see "Access Denied" instead of clean hiding). **Recommendation:** Override `_visible_menu_ids` for menu hiding PLUS use `res.groups` for security enforcement (defense in depth).

Create XML data files in `ncollection_core` defining `res.groups` for standard SaaS tenant roles. Groups: (1) `NCollection / Owner` — full control including billing access, implies all other `NCollection` groups + `base.group_system`. (2) `NCollection / CEO` — all ERP modules, read-only financials. (3) `NCollection / Manager` — department-level access. (4) `NCollection / Sales` — CRM + Sales + Invoicing, implies `sales_team.group_sale_salesman`. (5) `NCollection / Warehouse` — Inventory + Purchase, implies `stock.group_stock_user`. (6) `NCollection / HR` — HR module, implies `hr.group_hr_user`. (7) `NCollection / Accountant` — Accounting + Reports, implies `account.group_account_user`.

P1- Tenant Role
T07 Definitions

(8) `NCollection / Employee` — limited self-service (attendance, leave requests). Each group must use `implied_ids` correctly to inherit from base Odoo groups. (9) Create a role matrix documentation file. **Reason:** Predefined roles simplify tenant admin onboarding — they select from sensible defaults instead of understanding Odoo's complex permission system.

[DEV-2] P1-T03

Benefits: Consistent security across all tenants; faster onboarding; prevents misconfiguration. **Risks:** `implied_ids` chains can cause unintended permission escalation if not carefully designed. **Alternative:** Let each tenant define their own roles (rejected: too complex for SMB target market; high support burden). **Recommendation:** Provide predefined roles as defaults; allow Owner role to create custom groups in Phase 2+.

Remove access to Odoo "Apps" and "Settings" menus for all tenant users except the Owner role. (1) Override `ir.ui.menu` access for `base.menu_management` (Apps) — restrict to `NCollection / Owner` group only. (2) Override `ir.ui.menu` access for `base.menu_administration` (Settings) — restrict to Owner only. (3) Block direct URL access: override the Settings controller to return 403 for non-Owner users. (4) Disable Developer Mode toggle for non-Owner users. (5) Block `/web/database/manager` URL entirely (no user should access this). (6) Block debug mode activation via URL parameter (`?debug=1`). (7) Test: a Sales-role user must not see Apps or Settings menus; manually navigating to `/odoo/settings` must show Access Denied. **Reason:** Tenant users must not install/uninstall modules, change system settings, or access developer tools. These actions could break their workspace and create support tickets. **Benefits:** Prevents accidental or

P1- Apps &
T08 Settings Menu
Stripping

[DEV-2] P1-T07

malicious system modification by tenants; reduces support burden. **Risks:** Some legitimate Settings sub-menus (e.g., user management) may need to be exposed to Owner. Mitigation: create a simplified "Workspace Settings" view for Owner. **Recommendation:** Block everything first, then selectively expose Owner-appropriate settings through a custom menu.

Extend the EXISTING ncollection_branding module (currently: browser title, favicon, SCSS colors). Complete all remaining branding items: (1) Replace the Odoo logo in the top-left navbar with NCollection logo — use OWL component patching or QWeb template inheritance targeting the WebClient/NavBar. (2) Replace the loading/splash screen animation with NCollection branding. (3) Override "Powered by Odoo" footer text in all views. (4) Replace the About dialog (Help → About) — show NCollection ERP version, copyright, and company info instead of Odoo's. (5) Replace the default backend wallpaper/background. (6) Customize error pages (404, 500) with NCollection styling. (7) Ensure ALL text references to "Odoo" in the visible UI are replaced or hidden. (8) Audit: grep the entire rendered frontend for the string "Odoo" — anything visible must be replaced. **Reason:** The end-user experience must be 100% NCollection-branded. Any Odoo reference breaks the white-label illusion and may confuse tenants. **Benefits:** Professional product identity; client confidence; competitive differentiation. **Risks:** Some Odoo references may be deeply embedded in JavaScript bundles (e.g., error messages). Mitigation: override at the template level where possible; CSS display:none for stubborn references. **Recommendation:** Prioritize user-visible elements first; schedule a "branding audit" at the end of Phase 1 to catch remaining references.

[DEV-3] P1-T03

P1- Web Client
T09 Branding
Completion

Completely redesign the Odoo login page (/web/login) using QWeb template inheritance on web.login. (1) Custom full-page background (gradient or professional image). (2) NCollection logo prominently displayed above the login form. (3) Modern, centered login card with email/password fields. (4) "Remember Me" checkbox. (5) "Forgot Password" link (functional — sends reset email). (6) NCollection copyright footer. (7) Remove all Odoo references from the page HTML source. (8) If per-tenant login pages are desired: detect subdomain and optionally show tenant logo alongside NCollection logo. (9) Mobile-responsive design. **Reason:** The login page is the first impression. A generic Odoo login undermines the premium SaaS positioning. **Benefits:** Professional first impression; brand reinforcement on every login. **Risks:** Custom login page must not break Odoo's CSRF protection or session management. Mitigation: only override the template, not the controller logic. **Recommendation:** Override the template only; keep the controller intact to preserve security. Replace Odoo's default URL paths that contain "odoo" in the

[DEV-3] P1-T09

P1- Login Page
T10 Redesign

P1- URL Path T11 Rewriting	path. In Odoo 19, the backend URL is /odoo/.... (1) Use Nginx rewrite rules to map: /odoo → /app or root /. (2) Ensure internal Odoo redirects still work (may need JavaScript route override). (3) Ensure bookmarked old URLs still work (301 redirect from /odoo/... to new paths). (4) Test: no URL in the browser address bar should contain the word "odoo" during normal use. Reason: URL paths containing "odoo" reveal the underlying technology and undermine the white-label positioning. Benefits: Complete white-label; clean URLs. Risks: Odoo 19 may generate internal URLs with /odoo/ prefix that break if rewritten. Mitigation: only rewrite user-visible URLs; preserve internal API paths. Alternative: Accept /odoo/ URLs (rejected: undermines white-label goal). Recommendation: Implement Nginx-level rewrites first; evaluate the need for JavaScript-level route overrides after testing.	[DEV-3] P1-T09
P1- Dynamic T12 Tenant Branding	Build per-tenant visual customization. (1) Extend res.company with fields: nc_primary_color, nc_secondary_color, nc_sidebar_color, nc_login_background (Binary/Image). (2) Create a QWeb template (or OWL component) that generates a <style> block with CSS custom properties: --nc-primary, --nc-secondary, --nc-sidebar-bg, etc. (3) Inject this style block on every page load using session-based data. (4) Update the ncollection_branding SCSS to use these CSS variables with NCollection defaults as fallback. (5) Create a simple "Workspace Appearance" settings page for the tenant Owner to change colors and upload a logo. (6) Test: change a tenant's primary color → navbar/sidebar color updates on next page load. Reason: Tenant branding is a premium SaaS feature that increases perceived value and tenant satisfaction. Benefits: Tenants feel ownership of their workspace; reduces "generic SaaS" perception. Risks: CSS variable injection must be sanitized — prevent XSS via malicious color values. Mitigation: validate hex color format server-side. Recommendation: Implement with server-side validation; limit customization to colors and logo initially.	[DEV-3] P1-T05, P1-T09
P1- Customer T13 Authentication Hardening	Override the Odoo login controller (web.Home.web_login) to add SaaS-specific authentication: (1) Enforce that users can only log in to their assigned tenant database — the DB is determined by subdomain, never by user choice. (2) Implement login attempt rate limiting (max 5 failed attempts → 15-minute lockout per IP). (3) Add session timeout configuration (configurable per tenant via ir.config_parameter). (4) Implement "Forgot Password" flow with secure email reset tokens (time-limited, single-use). (5) Log all authentication events to ncollection.auth.log model (login success, login failure, logout, password reset). (6) Ensure session cookies have Secure, HttpOnly, and SameSite=Lax flags. Reason: Default Odoo authentication lacks rate limiting, session management, and audit logging — insufficient for a commercial SaaS platform. Benefits: Protection against brute-	[DEV-1] P1-T05

force attacks; audit compliance; configurable session policies per tenant. **Risks:** Overriding the login controller is sensitive — bugs here lock users out. Mitigation: extensive testing; feature flag to disable custom auth. **Alternative:** Use an external auth provider (Keycloak, Auth0) — rejected for Phase 1 (adds infrastructure complexity); reconsider in Phase 10.

Recommendation: Implement custom auth hardening with a feature flag toggle for safe rollout.

Build the main ERP landing dashboard for tenant users using the OWL framework. This is the CUSTOMER dashboard (what a CEO/Manager sees after login) — NOT the SaaS admin dashboard (which already exists). Widgets: (1) Sales summary card (this month vs. last month, with trend arrow), (2) Accounts Receivable total, (3) Accounts Payable total, (4) Cash/bank balance, (5) Open activities/tasks count, (6) Pending approvals count, (7) Quick action buttons (New Quotation, New Invoice, New Purchase Order), (8) Revenue trend chart (line chart, last 6 months — use Chart.js or ApexCharts), (9) Top 5 customers by revenue (bar chart). Each widget must fetch data via `useService('rpc')` from backend computed fields. Dashboard must be responsive for tablet screens. Dashboard must respect role permissions (Accountant sees financial widgets; Sales sees sales widgets; CEO sees everything). **Reason:** The default Odoo home page is a list of installed apps. A proper dashboard provides immediate business value upon login. **Benefits:** Executive visibility; faster decision-making; professional SaaS experience. **Risks:** Dashboard queries across multiple models can be slow if not optimized. Mitigation: use precomputed fields or caching for expensive aggregations. **Recommendation:** Start with simple `search_count` and `read_group` queries; optimize to raw SQL only if performance is insufficient.

Override all default Odoo email templates to use NCollection branding. (1) Create a base HTML email layout template: NCollection logo header, brand color accents, professional footer with company info. (2) Override: password reset emails, user invitation emails, quotation/SO emails, invoice emails, purchase order emails. (3) Remove all Odoo references from email HTML source. (4) Ensure responsive design for mobile email clients (Gmail, Outlook). (5) Test: send a sample email from each workflow and verify branding is correct. **Reason:** Emails are a high-visibility touchpoint. Odoo-branded emails undermine the white-label positioning. **Benefits:** Consistent brand experience; professional communication. **Risks:** Odoo email templates use complex QWeb inheritance — overriding can be fragile across Odoo upgrades. Mitigation: override at the highest-level template possible. **Recommendation:** Create one base email layout; have all email templates inherit from it. Comprehensive testing of all Phase 1 deliverables working together. (1) Create 2+ test tenant databases with different subscription plans. (2) Verify multi-tenant routing (access each

P1- Customer
T14 Workspace
Dashboard

[DEV-3] P1-T03

P1- Email
T15 Template
Branding

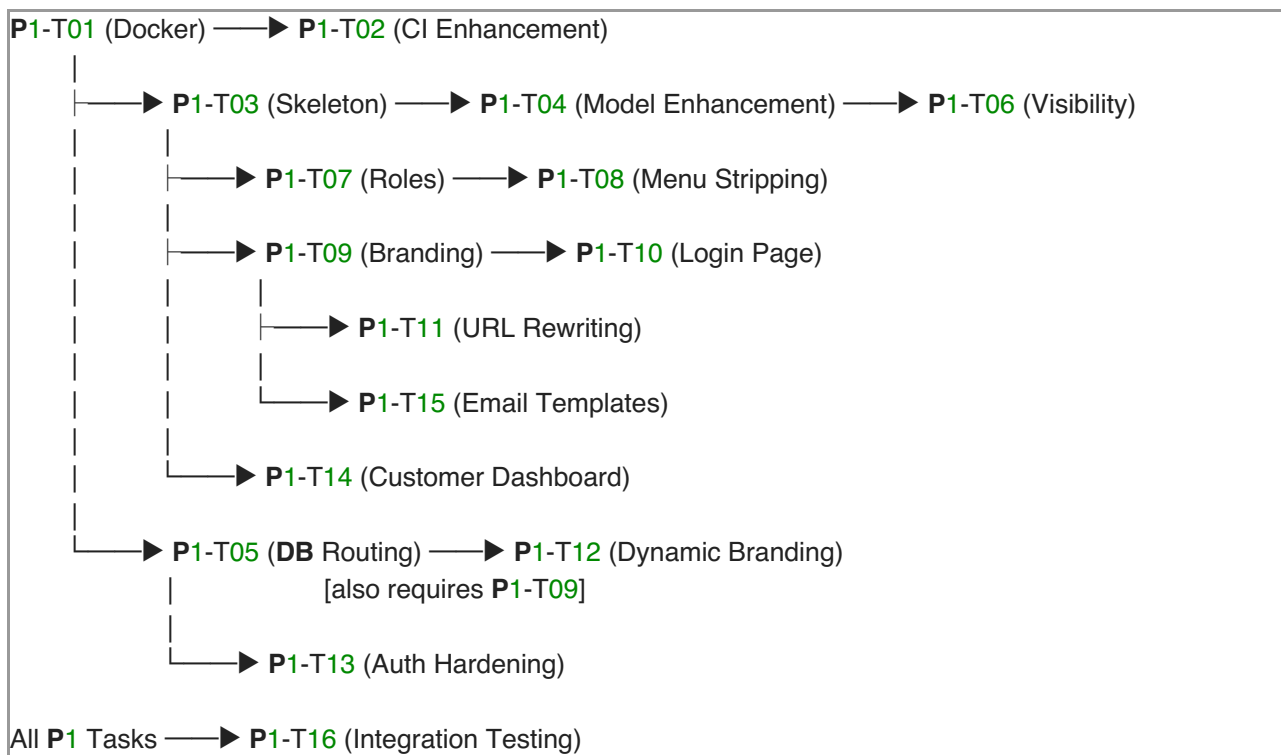
[DEV-3] P1-T09

P1- Phase 1
T16 Integration
Testing

tenant via different subdomain). (3) Verify module visibility (Starter tenant sees fewer menus than Enterprise tenant). (4) Verify role-based access (test each of the 8 roles). (5) Verify branding completeness (audit for any remaining "Odo" references). (6) Verify login page branding and functionality. (7) Verify dashboard loads with correct data per role. (8) Verify email templates send with correct branding. (9) Verify cross-tenant isolation (attempt to access another tenant's data — must fail). (10) Document results in a test report. Create regression test checklist for future sprints. **Reason:** Individual features may work in isolation but fail when combined. Integration testing catches gaps in tenant isolation, role enforcement, and branding. **Benefits:** Confidence before deploying to staging; documented regression checklist. **Risks:** Integration testing is time-consuming. Mitigation: automate repeatable tests where possible. **Recommendation:** Spend 2 full days on integration testing; document every test case and result.

[DEV-1] All P1 tasks

Phase 1 Dependency Graph



Phase 1 Developer Workload

Developer	Tasks	Total Days	Notes
DEV-1	P1-T01, T02, T03, T05, T13, T16	19	Heaviest load in Week 1 (infra setup); then routing + auth
DEV-2	P1-T04, T06, T07, T08	15	Blocked until P1-T03 complete; start with role design analysis
DEV-3	P1-T09, T10, T11, T12,	23	Heaviest overall; consider shifting P1-T15 to DEV-2 if

Phase 2: SaaS Automation

Priority: P1 — HIGH (starts after Phase 1 stabilization)

Objective: Automate tenant provisioning, billing, subscription lifecycle, backups, and domain management.

This phase builds on the SaaS foundation models (already completed) and the Customer Workspace (Phase 1). It transforms manual operations into automated pipelines.

Phase 2 Tasks

ID	Task Name	Description	Assigned Dependencies	Est. Days
P2-T01	Provisioning Engine Activation	Extend the EXISTING ncollection.provisioning.job model into a fully functional provisioning engine. action_run_provisioning(): (1) Create new PostgreSQL database via Odoo CLI (odoo-bin -d {db_name} -i base --stop-after-init) or direct SQL CREATE DATABASE, (2) Install subscription-specified modules, (3) Create tenant admin user, (4) Write ncollection.workspace.config record with allowed modules and plan info, (5) Apply branding defaults, (6) Update job status through each stage, (7) Rollback on failure (drop DB). Before building: Check OCA/queue → queue_job for async job processing. Reason: Manual DB creation doesn't scale beyond 10 tenants. Benefits: One-click tenant onboarding; consistent setup. Risks: odoo-bin CLI execution from within a running Odoo process may cause resource contention. Alternative: Use subprocess.Popen with resource limits. Recommendation: Use queue_job OCA module for async execution with retry logic.	[DEV-1] P1-T04	5
	Auto-Provisioning Pipeline	End-to-end automation: subscription draft→active triggers provisioning. (1) Auto-generate sanitized DB name from company name, (2) Create provisioning job, (3) Execute async (via queue_job), (4) On success: update tenant status, set portal_url, send welcome email, (5) On failure: alert admin, log details. Add "Provision Now" button on Tenant form. Reason: Eliminates manual steps in tenant creation. Benefits: Sub-10-minute tenant onboarding. Risks: Async failure may leave partial state. Recommendation: Implement idempotent provisioning (safe to retry). Automated backup system. Create ncollection.backup model. Daily ir.cron: (1) pg_dump --format=custom each active tenant DB, (2) Compress + encrypt, (3) Upload to S3/Backblaze B2, (4) Apply retention: 7 daily, 4 weekly, 12 monthly, (5) Log results, alert on failures. Add	[DEV-1] P2-T01	5

P2- T03	Backup Manager	"Restore Backup" wizard. Before building: Check OCA for backup modules. Reason: Data loss is catastrophic for SaaS. Benefits: Automated, tested backups; tenant-level restore. Risks: pg_dump of large DBs can take minutes; schedule during off-peak. Recommendation: Run backups between 02:00–05:00 UTC; test restore monthly. Automate subdomain and SSL. On provisioning: (1) Generate Nginx server block from Jinja2 template, (2) Reload Nginx gracefully, (3) Request Let's Encrypt cert via Certbot, (4) Store domain + SSL expiry on tenant record. Create ncollection.domain model. Weekly cron: auto-renew certificates expiring within 14 days. Reason: Manual domain/SSL management doesn't scale. Benefits: Instant tenant availability; zero SSL lapses. Risks: Let's Encrypt rate limits (50 certs/week). Recommendation: Use wildcard cert initially; per-tenant certs for custom domains. Daily ir.cron: (1) Find subscriptions where end_date < today, (2) Transition to expired, (3) Update tenant status, (4) Send expiration email, (5) After grace period (15 days): transition to suspended — show "Subscription Expired" page. Add advance warning emails at 30, 14, 7, 1 day before expiry. Reason: Manual tracking of expiring subscriptions is unsustainable. Benefits: Automated lifecycle management; reduced churn via reminders. Risks: Incorrect expiration could lock out paying customers. Recommendation: Add 48-hour grace period buffer; admin override for reactivation. Auto-generate Odoo invoices when subscriptions are purchased/renewed. (1) Create account.move in admin DB, (2) Line items: plan name, period, price, (3) Apply UAE VAT (5%), (4) On upgrade: prorate difference. Link invoices to tenant. Track payment status. Before building: Check OCA for subscription billing modules. Reason: Manual invoicing doesn't scale. Benefits: Automated revenue tracking. Risks: VAT calculation must be accurate for UAE compliance. Recommendation: Implement basic invoicing first; add payment gateway integration in Phase 6. Complete lifecycle: draft→active→expired→suspended→terminated. Also active→cancelled. Each transition triggers actions: draft→active: provision + invoice; expired→suspended: block access; suspended→active: reactivation flow. Add action_renew(), action_upgrade(), action_downgrade(). Reason: Formal state machine prevents invalid transitions and ensures consistent side effects. Benefits: Predictable behavior; audit trail. Risks: Edge cases (e.g., reactivation during provisioning) need careful handling. Recommendation: Implement with _check_	[DEV-1]	P2-T02	4
P2- T04	Domain & SSL Manager	Manual domain/SSL management doesn't scale. Benefits: Instant tenant availability; zero SSL lapses. Risks: Let's Encrypt rate limits (50 certs/week). Recommendation: Use wildcard cert initially; per-tenant certs for custom domains. Daily ir.cron: (1) Find subscriptions where end_date < today, (2) Transition to expired, (3) Update tenant status, (4) Send expiration email, (5) After grace period (15 days): transition to suspended — show "Subscription Expired" page. Add advance warning emails at 30, 14, 7, 1 day before expiry. Reason: Manual tracking of expiring subscriptions is unsustainable. Benefits: Automated lifecycle management; reduced churn via reminders. Risks: Incorrect expiration could lock out paying customers. Recommendation: Add 48-hour grace period buffer; admin override for reactivation. Auto-generate Odoo invoices when subscriptions are purchased/renewed. (1) Create account.move in admin DB, (2) Line items: plan name, period, price, (3) Apply UAE VAT (5%), (4) On upgrade: prorate difference. Link invoices to tenant. Track payment status. Before building: Check OCA for subscription billing modules. Reason: Manual invoicing doesn't scale. Benefits: Automated revenue tracking. Risks: VAT calculation must be accurate for UAE compliance. Recommendation: Implement basic invoicing first; add payment gateway integration in Phase 6. Complete lifecycle: draft→active→expired→suspended→terminated. Also active→cancelled. Each transition triggers actions: draft→active: provision + invoice; expired→suspended: block access; suspended→active: reactivation flow. Add action_renew(), action_upgrade(), action_downgrade(). Reason: Formal state machine prevents invalid transitions and ensures consistent side effects. Benefits: Predictable behavior; audit trail. Risks: Edge cases (e.g., reactivation during provisioning) need careful handling. Recommendation: Implement with _check_	[DEV-1]	P1-T05	3
P2- T05	Subscription Expiration Scheduler	Expired" page. Add advance warning emails at 30, 14, 7, 1 day before expiry. Reason: Manual tracking of expiring subscriptions is unsustainable. Benefits: Automated lifecycle management; reduced churn via reminders. Risks: Incorrect expiration could lock out paying customers. Recommendation: Add 48-hour grace period buffer; admin override for reactivation. Auto-generate Odoo invoices when subscriptions are purchased/renewed. (1) Create account.move in admin DB, (2) Line items: plan name, period, price, (3) Apply UAE VAT (5%), (4) On upgrade: prorate difference. Link invoices to tenant. Track payment status. Before building: Check OCA for subscription billing modules. Reason: Manual invoicing doesn't scale. Benefits: Automated revenue tracking. Risks: VAT calculation must be accurate for UAE compliance. Recommendation: Implement basic invoicing first; add payment gateway integration in Phase 6. Complete lifecycle: draft→active→expired→suspended→terminated. Also active→cancelled. Each transition triggers actions: draft→active: provision + invoice; expired→suspended: block access; suspended→active: reactivation flow. Add action_renew(), action_upgrade(), action_downgrade(). Reason: Formal state machine prevents invalid transitions and ensures consistent side effects. Benefits: Predictable behavior; audit trail. Risks: Edge cases (e.g., reactivation during provisioning) need careful handling. Recommendation: Implement with _check_	[DEV-1]	P1-T04	2
P2- T06	Billing Engine	invoices to tenant. Track payment status. Before building: Check OCA for subscription billing modules. Reason: Manual invoicing doesn't scale. Benefits: Automated revenue tracking. Risks: VAT calculation must be accurate for UAE compliance. Recommendation: Implement basic invoicing first; add payment gateway integration in Phase 6. Complete lifecycle: draft→active→expired→suspended→terminated. Also active→cancelled. Each transition triggers actions: draft→active: provision + invoice; expired→suspended: block access; suspended→active: reactivation flow. Add action_renew(), action_upgrade(), action_downgrade(). Reason: Formal state machine prevents invalid transitions and ensures consistent side effects. Benefits: Predictable behavior; audit trail. Risks: Edge cases (e.g., reactivation during provisioning) need careful handling. Recommendation: Implement with _check_	[DEV-2]	P1-T04	5
P2- T07	Subscription Lifecycle State Machine	block access; suspended→active: reactivation flow. Add action_renew(), action_upgrade(), action_downgrade(). Reason: Formal state machine prevents invalid transitions and ensures consistent side effects. Benefits: Predictable behavior; audit trail. Risks: Edge cases (e.g., reactivation during provisioning) need careful handling. Recommendation: Implement with _check_	[DEV-2]	P2-T06, P1-T04	4

		constraints to validate state transitions.			
		Extend EXISTING dashboard transient model. Add views: (1) KPI summary (total tenants, MRR, churn rate, trial conversion), (2) Tenant list with color-coded status badges + quick actions, (3) Provisioning job log with status filters, (4) Revenue analytics (monthly trend, per-plan breakdown), (5) System health (DB sizes, storage).	[DEV-2]	P1-T04	4
P2-T08	SaaS Admin Dashboard Enhancement	Restrict to NCollection / Platform Admin group. Reason: Platform operators need visibility into business and operational metrics. Benefits: Data-driven operations; proactive issue detection. Recommendation: Implement incrementally — KPI cards first, then charts.			
		Public-facing subscription purchase pages. (1) Landing page with plan comparison table, (2) Billing cycle toggle (monthly/yearly), (3) Company registration form, (4) Payment placeholder (actual gateway in Phase 6), (5) "Your workspace is being prepared" confirmation. On submit: create tenant + subscription, trigger provisioning.			
P2-T09	Public Checkout Flow	All pages: NCollection branding, zero Odoo references. Reason: Self-service signup is essential for SaaS scalability. Benefits: 24/7 customer acquisition without staff involvement. Risks: Public forms need strong input validation and spam protection. Recommendation: Add reCAPTCHA; validate company name uniqueness; sanitize all inputs.	[DEV-3]	P1-T09	5
		Complete transactional email set: (1) Welcome (on provisioning complete — login URL, credentials, getting-started guide), (2) Renewal reminders (30/14/7/1 day), (3) Expiration notice, (4) Suspension warning, (5) Payment confirmation, (6) Upgrade/downgrade confirmation. Use NCollection email layout from P1-T15.			
P2-T10	Email Automation System	Schedule emails to avoid sending multiple on the same day. Reason: Lifecycle emails reduce churn and improve tenant retention. Benefits: Automated communication; professional customer journey. Recommendation: Use ir.cron with mail.mail for scheduled sending.	[DEV-3]	P1-T15, P2-T05	3

Phase 3: ERP Enhancement & UAE Localization

Priority: P1 — HIGH (can overlap with Phase 2 for DEV-2 and DEV-3)

Objective: Enhance core ERP modules and implement full UAE/GCC localization.

Phase 3 Tasks

ID	Task Name	Description	Assigned	Dependencies	Est.
P3-	OCA Financial Stack	Verify EXISTING OCA modules (account_financial_report, mis_builder) work correctly with tenant provisioning. Ensure they are auto-installed during provisioning. Test all report types with UAE	[DEV-1]	P1-T01	2

T01 Verification	sample data. Patch if needed for Odoo 19 compatibility. Before building: Check OCA 19.0 branches for updates.				
P3-T02 PostgreSQL Performance Tuning	Tune postgresql.conf for multi-tenant ERP: shared_buffers (25% RAM), effective_cache_size (75%), work_mem (64MB), maintenance_work_mem (512MB), SSD settings. Create indexes on hot fields. Setup pg_stat_statements. Log slow queries (>500ms). Tune odoo.conf workers. Template config for env-variable substitution. Load test with simulated concurrent users. Document performance baseline and scaling thresholds.	[DEV-1]	P1-T01	2	
P3-T03 Odoo Worker Optimization	Create ncollection_uae addon. Tax records: 5% standard, 0% zero-rated, exempt. Tax groups, fiscal positions (domestic/GCC/international). Default taxes on products. All via XML data files for auto-install.	[DEV-1]	P3-T02	2	
P3-T04 UAE VAT Configuration	Standard UAE CoA within ncollection_uae: Assets (1xxx), Liabilities (2xxx), Equity (3xxx), Revenue (4xxx), COGS (5xxx), Expenses (6xxx-7xxx). Tax accounts linked to P3-T04. Test: complete sale→invoice→payment cycle with correct journal entries.	[DEV-2]	P1-T03	3	
P3-T05 UAE Chart of Accounts	AED as default. Enable multi-currency. Auto exchange rates (ECB/UAE Central Bank). Common GCC currencies: USD, EUR, SAR, KWD, BHD, QAR, OMR. Rounding rules per UAE standards.	[DEV-2]	P3-T01, P3-T04	4	
P3-T06 AED Currency & Multi-Currency	Multi-level approvals for UAE businesses. Sales: manager approval above threshold. Purchase: two-level (dept manager + finance). CRM: territory-based lead assignment. Using mail.activity and custom state fields — NOT core workflow modification.	[DEV-2]	P3-T04	2	
P3-T07 Workflow Enhancements	Complete bilingual support. Export .po files for all ncollection_* modules. Translate all labels, menus, status values, error messages. Verify RTL layout. Test Arabic PDF rendering (font support).	[DEV-2]	P1-T07	5	
P3-T08 Arabic/English Translation	Custom QWeb PDF templates: bilingual header, TRN, itemized VAT, QR code (e-invoicing readiness), NCollection or tenant branding, sequential numbering, bank details. A4 + thermal receipt formats.	[DEV-3]	P1-T09	5	
P3-T09 UAE-Compliant PDF Invoices	Extend EXISTING ncollection_mis_templates: Balance Sheet (proper account grouping), P&L, Cash Flow (if feasible). Add period comparison, budget vs. actual. Verify with UAE CoA.	[DEV-3]	P3-T04	4	
P3-T10 MIS Builder Report Enhancement		[DEV-3]	P3-T01, P3-T05	3	

Phase 4: Executive Dashboards

Priority: P2 — MEDIUM

Objective: Real-time analytics dashboards for tenant executives and department managers.

ID	Task	Assigned	Dependencies	Est.
P4-T01	Data Aggregation Engine	Optimized SQL/Python for cross-module data aggregation within a tenant. Caching layer for expensive queries.	[DEV-1]	P1-T04 4
P4-T02	KPI Logic Models	ncollection.kpi model. Computed KPIs: Revenue Growth %, Avg Deal Size, DSO, Gross Margin %, Employee Turnover, Inventory Turnover.	[DEV-2]	P4-T01 3
P4-T03	CEO Dashboard UI	OWL dashboard: KPI cards, revenue trend chart, sales pipeline, top customers. Chart.js/ApexCharts. Date range selector. Drill-down navigation. < 3s load time.	[DEV-3]	P4-T02 5
P4-T04	Department Dashboards	Role-specific: Sales (pipeline, targets), Finance (receivables aging, P&L sparkline), HR (headcount, leave), Warehouse (stock valuation, low-stock alerts).	[DEV-3]	P4-T02 5

Phase 5: AI Platform

Priority: P3

Objective: AI-powered assistance for ERP users.

ID	Task	Assigned	Dependencies	Est.
P5-T01	LLM Gateway	Secure API gateway to OpenAI/Claude. Tenant-scoped rate limiting. Token usage tracking. API key management in ir.config_parameter.	[DEV-1]	P1-T04 4
P5-T02	Context Injection Engine	Tenant-specific data enrichment for LLM prompts. PII sanitization. Context window management. Prompt templates. Absolute tenant isolation.	[DEV-1]	P5-T01, P4-T01 5
P5-T03	Anomaly Detection	Background jobs: stock below safety level, sales drop detection, unusual expenses, attendance anomalies. Alert records with severity and suggested actions.	[DEV-2]	P4-T01 4
P5-T04	NL→Domain Mapper	Natural language to Odoo domain filter translation via LLM. Input validation. Supported models: sale.order, account.move, stock.picking, crm.lead.	[DEV-2]	P5-T01 5
P5-T05	AI Chat Widget	Persistent OWL floating chat widget. Message history. Markdown rendering. Suggested prompts. Minimize/maximize.	[DEV-3]	P5-T02 5
P5-T06	Smart Search UI	NL-enhanced Odoo search bar. Model-grouped results dropdown. AI search toggle. Recent query cache.	[DEV-3]	P5-T04 4

Phase 6: Customer Portal

Priority: P3

Objective: Self-service portal for the tenants' end-customers.

ID	Task	Assigned	Dependencies	Est.
P6-T01	Payment Gateways Portal	Stripe + PayTabs + Tap Payments integration. Webhook handling. Auto-reconciliation. PCI-DSS compliance (tokenization). Check OCA first: OCA/payment for existing providers.	[DEV-1]	P2-T06 5

P6- Access	Strict ir.rule for portal users. Own invoices, orders, tickets only.	[DEV-2]	P1- 3
T02 Rights	Extensive isolation testing.		T07
P6- Support	ncollection.support.ticket model. Portal submission form. Auto-assignment. SLA tracking. CSAT rating. Check OCA first:	[DEV-2]	P6- 5
T03 Ticketing	OCA/helpdesk.		T02
P6- Portal UI	Override /my portal templates. Modern card-based design. Tenant branding (not NCollection branding). Responsive.	[DEV-3]	P6- 5
T04 Redesign			T02
P6- Knowledge	ncollection.knowledge.article model. Category navigation. Full-text	[DEV-3]	P6- 4
T05 Base	PostgreSQL tsvector search. Admin editor.		T04

Phase 7: Mobile Application

Priority: P3

Objective: Mobile accessibility for field workers and executives.

ID	Task	Assigned	Dependencies	Est.
P7-T01	Mobile API Optimization	Lightweight JSON-RPC wrappers. Pagination. Field selection. Compression. JWT auth tokens. API versioning.	[DEV-1]	P1-T015
P7-T02	Push Notification Server	Firebase FCM integration. Device registration model. Triggers: approval requests, lead assignment, stock alerts, payment received.	[DEV-1]	P7-T013
P7-T03	Offline Sync Logic	Conflict resolution: last-write-wins (simple fields), server-wins (financial), merge (chatter). Sync queue model.	[DEV-2]	P7-T015
P7-T04	Barcode Endpoints	Hyper-optimized endpoints: scan, transfer, receive, pick. < 200ms response. Redis cache for product data.	[DEV-2]	P7-T014
P7-T05	Mobile App Scaffold	React Native or Flutter project. Auth (login + biometrics). API layer. State management. Offline storage. Push notification handler.	[DEV-3]	P7-T015
P7-T06	Mobile UI Screens	Dashboard, Sales Entry, Barcode Scanner, Approvals, Notifications, Profile. NCollection + tenant branding.	[DEV-3]	P7-T058

Phase 8: Platform Services

Priority: P3

Objective: Enterprise integrations and operational monitoring.

ID	Task	Assigned	Dependencies	Est.
P8- T01	Public REST API	OAuth2 authentication. Rate limiting. Standard REST endpoints for Contacts, Products, Sales, Invoices, Inventory. Versioned /api/v1/. Check OCA first: OCA/rest-framework → base_rest.	[DEV-1]	P1- T01 8
	P8- T02	Webhooks System	Event-driven outgoing webhooks. HMAC-SHA256 signing. Retry with backoff. Event catalog: sale/invoice/stock/crm events. Check OCA first: OCA/server-tools → base_webhook.	[DEV-1] P8- T01 4
P8- T03	System Monitoring	Prometheus + Grafana. node_exporter, postgres_exporter, custom Odoo metrics. Alert rules: CPU, disk, connections, SSL expiry.	[DEV-1]	P1- T01 3
P8-		Field-level change tracking on critical models.		P1-

T04	Audit Trail	ncollection.audit.log model. Per-record history tab. CSV export. Retention policy. Check OCA first: OCA/server-tools → auditlog.	[DEV-2]	T04	4
P8- T05	Developer SDK	Python + Node.js client SDKs. Auto-generated from OpenAPI spec. Documentation site.	[DEV-2]	P8- T01	5
P8- T06	API Documentation Portal	Swagger UI or Redoc. Auto-generated OpenAPI 3.0 spec. "Try it out" sandbox. NCollection branding.	[DEV-3]	P8- T01	3
P8- T07	Integration Directory UI	Browse/install verified integrations. ncollection.marketplace.listing model. Category filters. Admin curation.	[DEV-3]	P1- T03	5

Phase 9: Marketplace

Priority: P3

Objective: Build a full-featured marketplace for third-party addon distribution.

ID	Task	Assigned	Dependencies	Est.
P9-T01	Marketplace Backend	ncollection.marketplace.app model with versioning, compatibility matrix, pricing, reviews. Developer submission workflow. Automated compatibility testing pipeline. App signing for trust verification.	[DEV-1]	P8-T018
P9-T02	App Installation Engine	Secure server-side module installation from marketplace. Sandbox testing before install. Rollback on failure. Per-tenant installation tracking. Dependency resolution.	[DEV-1]	P9-T015
P9-T03	Developer Portal	Self-service portal for third-party developers. App submission form. Documentation requirements. Review process. Revenue sharing configuration. Analytics dashboard (installs, ratings, revenue).	[DEV-2]	P9-T015
P9-T04	App Review & Rating System	Customer reviews and ratings. Review moderation. Average rating computation. Featured/trending algorithms. Abuse detection.	[DEV-2]	P9-T013
P9-T05	Marketplace Storefront UI	Public-facing marketplace website. Category browsing, search, detail pages, screenshots, reviews, install button. NCollection branding. SEO-optimized.	[DEV-3]	P9-T016
P9-T06	In-App Marketplace Widget	OWL component within the tenant ERP. Browse marketplace from within their workspace. One-click install. Installed apps management.	[DEV-3]	P9-T024

Phase 10: Enterprise Readiness

Priority: P3

Objective: Harden the platform for enterprise-scale operations and compliance.

ID	Task	Assigned	Dependencies	Est.	
P10-T01	High Availability Setup	Multi-server deployment with load balancing. PostgreSQL streaming replication (primary + standby). Automated failover. Zero-downtime deployments via blue-green or rolling updates.	[DEV-1]	P8-T03	8
P10-T02	Horizontal Scaling	Docker Swarm or Kubernetes orchestration. Auto-scaling Odoo workers based on request load. Shared filestore (NFS or S3-backed). Session stickiness or Redis-backed sessions. External auth provider integration (Keycloak/Auth0). Two-factor	[DEV-1]	P10-T01	6

P10-T03	Advanced Security	authentication (TOTP). IP whitelisting per tenant. SOC 2 compliance checklist. Penetration testing. Security audit.	[DEV-1]	P1-T13	5
P10-T04	Multi-Region Support	Geo-distributed deployment for GCC coverage. Data residency per region (UAE data stays in UAE). CDN for static assets. Region-aware DNS routing.	[DEV-1]	P10-T02	6
P10-T05	Enterprise Accounting	Advanced financial workflows: multi-company consolidation, intercompany transactions, advanced bank reconciliation. Evaluate Enterprise-level OCA modules.	[DEV-2]	P3-T05	5
P10-T06	Compliance & Governance	UAE e-invoicing compliance (when mandated). GDPR-equivalent data protection. Tenant data export (right to portability). Tenant data deletion (right to erasure). Data retention policies.	[DEV-2]	P8-T04	4
P10-T07	Enterprise Onboarding Wizard	Guided setup wizard for new enterprise tenants. Industry-specific templates (trading, services, manufacturing). Data migration tools (import from Excel/CSV/other ERPs). Setup checklist with progress tracking.	[DEV-3]	P2-T02	5
P10-T08	White-Label Reseller System	Allow NCollection partners to resell the platform under their own brand. Partner dashboard. Revenue sharing. Sub-tenant management. Partner-specific branding cascading.	[DEV-3]	P1-T12	6

18. Cross-Cutting Concerns

18.1 Testing Strategy

Level	Scope	Tools	Responsibility
Unit Tests	Model methods, computed fields, constraints	odoo.tests.common.TransactionCase	Each DEV for own code
Integration Tests	Cross-model workflows, provisioning pipeline	odoo.tests.common.HttpCase	DEV-1 leads
Security Tests	Cross-tenant access, role enforcement	Manual + custom scripts	DEV-1 + DEV-2
UI Tests	Dashboard rendering, branding completeness	Manual + Odoo Tours	DEV-3 leads
Load Tests	Multi-tenant performance, concurrent users	Locust / k6	DEV-1
Regression Tests	After every sprint, verify no regressions	Automated test suite	All DEVs

18.2 Documentation Requirements

Every completed phase must produce:

1. **Technical Docs:** Model schemas, API specs, configuration guides
2. **User Guides:** How to use features (for tenant admins and end-users)
3. **Admin Guides:** How NCollection staff operate the platform
4. **Runbooks:** Operational procedures (backup, restore, scale, debug, incident response)

18.3 Environment Progression

Local (docker-compose.dev.yml)

- Staging (Hetzner VPS #1 — auto-deploy **on** merge **to** develop)
 - Production (Hetzner VPS #2 — manual promotion from main)
-

Document End

This is a living document. Update after each sprint to reflect completed tasks, new decisions, and architectural changes. Never redesign completed milestones unless explicitly requested.